```
was limited.",
"Loved the ambience, loved the food",
"The food is delicious but not over the top.",
"Service - Little slow, probably because too many people.",
"The place is not easy to locate",
"Mushroom fried rice was tasty"]

sid = SentimentIntensityAnalyzer()
for sentence in hotel_rev:
     print(sentence)
     ss = sid.polarity_scores(sentence)
     for k in ss:
         print('{0}: {1}, '.format(k, ss[k]), end='')
     print()
```

When running the above Python script, the different sentiment proportions for individual sentences are obtained as shown below:

```
Great place to be when you are in Bangalore.
neg: 0.0, neu: 0.661, compound: 0.6249, pos: 0.339,

The place was being renovated when I visited so the seating
was limited.
neg: 0.147, neu: 0.853, compound: -0.2263, pos: 0.0,

Loved the ambience, loved the food
neg: 0.0, neu: 0.339, compound: 0.8316, pos: 0.661,

The food is delicious but not over the top.
neg: 0.168, neu: 0.623, compound: 0.1184, pos: 0.209,

Service - Little slow, probably because too many people.
neg: 0.0, neu: 1.0, compound: 0.0, pos: 0.0,

The place is not easy to locate
neg: 0.286, neu: 0.714, compound: -0.3412, pos: 0.0,

Mushroom fried rice was tasty
neg: 0.0, neu: 1.0, compound: 0.0, pos: 0.0,
```

The compound value here conveys the overall positive or negative user experience.

Analysis using Naïve's Bayes Classifier

Apart from Vader, one can create one's own classification model using Naïve's Bayes Classifier. In the machine learning context, Naïve's Bayes Classifier is a probabilistic classifier based on Bayes' theorem that constructs a classification model out of training data. This classifier learns to classify the reviews to positive or negative using the supervised learning mechanism. The learning process starts by feeding in sample data that aids the classifier to construct a model to classify these reviews.

```
import nltk
from nltk.tokenize import word_tokenize

# Step 1 – Training data
train = [("Great place to be when you are in Bangalore.",
"pos"),
  ("The place was being renovated when I visited so the
seating was limited.", "neg"),
  ("Loved the ambience, loved the food", "pos"),
  ("The food is delicious but not over the top.", "neg"),
  ("Service - Little slow, probably because too many
people.", "neg"),
  ("The place is not easy to locate", "neg"),
  ("Mushroom fried rice was spicy", "pos"),
]

# Step 2
dictionary = set(word.lower() for passage in train for word
in word_tokenize(passage[0]))

# Step 3
t = [({word: (word in word_tokenize(x[0])) for word in
dictionary}, x[1]) for x in train]

# Step 4 – the classifier is trained with sample data
classifier = nltk.NaiveBayesClassifier.train(t)

test_data = "Manchurian was hot and spicy"
test_data_features = {word.lower(): (word in word_
tokenize(test_data.lower())) for word in dictionary}

print (classifier.classify(test_data_features))
```

The output for the above code can be 'pos', denoting positive. The training data here is an array of sentences with corresponding class types – positive (pos) or negative (neg) to train the classifier. The dictionary formed in Step 2 consists of all the words obtained by tokenising or breaking this list of sentences. Step 3 starts constructing the data to be fed to the Naïve Bayes Classifier and Step 4 feeds the data to the classifier. With these steps, you might try out testing the classifier with different sentences. END

References

[1] *http://www.nltk.org/howto/sentiment.html*
[2] *http://www.nltk.org/api/nltk.sentiment.html*
[3] Hutto, C.J. and Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Eighth International Conference on Weblogs and Social Media (ICWSM-14). Ann Arbor, MI, June 2014.

By: Shravan I.V.

The author currently works as a software developer at Cisco Systems India. He can be contacted at *iv.shravan@gmail.com*